

R pour jeunes économistes

Tidyverse pour la recherche empirique en économie

J.-B. Guiffard

15 mai 2026

R pour jeunes économistes

Pourquoi utiliser R en économie appliquée ?

R est aujourd'hui un outil central pour le travail empirique en économie.

Il permet de réaliser l'ensemble de la chaîne de traitement d'un projet de recherche :

données brutes → nettoyage → analyse → résultats → document final

Concrètement, R permet de :

- importer, nettoyer et organiser des données ;
- produire des statistiques descriptives et des graphiques ;
- estimer des modèles économétriques ;
- automatiser des tâches répétitives ;
- produire des documents reproductibles.

R n'est pas seulement un logiciel statistique : c'est un outil de travail pour construire une **recherche empirique reproductible**.

Une compétence utile pour la recherche

Pour un jeune économiste, apprendre R, ce n'est pas seulement apprendre un langage de programmation.

C'est apprendre à structurer un projet empirique de manière claire, transparente et réutilisable.

Un bon script doit permettre de comprendre :

- d'où viennent les données ;
- quelles transformations ont été appliquées ;
- comment les résultats ont été produits ;
- comment reproduire les tableaux, graphiques et estimations.

L'objectif de ce module : Vous donner les bases nécessaires pour utiliser R dans vos propres travaux de recherche, de manière autonome et progressive, et découvrir quelques packages “incontournables” ou pratiques dans votre recherche en économie.

Quelles sont vos attentes ?

Avant de commencer, prenons quelques minutes pour identifier vos besoins...

Quelques questions :

- Avez-vous déjà utilisé R ou RStudio ?
- Pour quels types de données souhaitez-vous utiliser R ?
- Quelles tâches vous posent aujourd'hui le plus de difficultés ?
- Avez-vous déjà utilisé le tidyverse ?
- Avez-vous déjà produit un document avec RMarkdown ?
- Qu'aimeriez-vous être capable de faire à la fin de la formation ?

Tidyverse

Objet de la séance

Cette première séance est consacrée à l'utilisation du **tidyverse** pour le traitement, l'exploration et la visualisation de données en économie appliquée.

L'objectif est de comprendre comment passer :

données brutes → base exploitable → statistiques descriptives → graphiques

Nous travaillerons à partir d'un exemple concret de base pays-années sur les émissions de CO2.

⇒ Le tidyverse permet d'écrire un code clair, lisible et reproductible pour construire une base d'analyse.

Plan de la séance

La séance est organisée en quatre étapes.

- 1 Comprendre le rôle du tidyverse dans un projet de recherche empirique
- 2 Importer et explorer une base de données
- 3 Manipuler les données avec dplyr
- 4 Produire des graphiques avec ggplot2

Nous terminerons par une discussion autour de vos propres matériaux de recherche. $\Rightarrow$ L'objectif est d'être capable de réutiliser les commandes vues aujourd'hui dans vos propres données de recherche.

Pourquoi le tidyverse ?

Qu'est-ce que le tidyverse ?

Le **tidyverse** est un ensemble de packages R conçus pour faciliter le travail sur les données.

Ces packages partagent une même logique :

- une syntaxe cohérente ;
- une manière commune de représenter les données ;
- une organisation du code lisible ;
- une logique adaptée aux étapes d'un projet empirique.

Le tidyverse permet de passer facilement des données brutes à une base exploitable pour l'analyse (plus simplement que le langage R de base).

Qu'est-ce que le tidyverse ?



Qu'est-ce que le tidyverse ?

Dans cette séance, nous utiliserons surtout quatre packages.

- 1 readr : importer des fichiers de données ;
- 2 dplyr : sélectionner, filtrer, transformer et résumer les données ;
- 3 tidyr : réorganiser les données ;
- 4 ggplot2 : produire des graphiques.

D'autres packages du tidyverse sont également utiles :

- tibble : travailler avec des tableaux de données modernes ;
- stringr : manipuler du texte ;
- forcats : manipuler des variables catégorielles ;
- purrr : automatiser des opérations répétitives.

Charger tidyverse permet de charger directement les principaux outils utilisés dans l'analyse de données.

Pourquoi est-ce utile pour un doctorant en économie ?

Dans un travail de recherche empirique, une grande partie du temps est consacrée aux données.

Avant d'estimer un modèle, il faut souvent :

- importer plusieurs bases ;
- comprendre leur structure ;
- nettoyer les variables ;
- créer des indicateurs ;
- agréger les données au bon niveau ;
- vérifier les valeurs manquantes ;
- produire des statistiques descriptives ;
- représenter graphiquement les principaux faits stylisés.

Le tidyverse est utile parce qu'il aide à organiser toutes ces étapes dans un code clair et reproductible.

Une logique de workflow empirique

L'objectif n'est pas seulement d'apprendre des fonctions. L'objectif est d'apprendre à construire une chaîne de traitement :

données brutes → données nettoyées → base d'analyse →
statistiques → graphiques → résultats

Avec le tidyverse, cette chaîne peut être écrite de manière lisible :

```
data_raw %>%  
  filter(condition) %>%  
  mutate(new_variable = transformation) %>%  
  group_by(group_variable) %>%  
  summarise(mean_variable = mean(variable, na.rm = TRUE))
```

(Le pipe %>% permet d'enchaîner plusieurs opérations de manière lisible.)

Le principe est qu'un bon code doit permettre de comprendre comment un résultat a été produit.

Un exemple de question empirique

Dans cette séance, nous travaillerons à partir d'une base sur les émissions de CO2.

Question simple :

Comment les émissions de CO2 par habitant ont-elles évolué selon les pays et les niveaux de développement ?

Un exemple de question empirique

Cette question permet d'illustrer plusieurs tâches classiques :

- sélectionner un échantillon d'analyse ;
- créer des variables par habitant ;
- comparer des pays ou des groupes de pays ;
- produire des statistiques descriptives ;
- représenter une évolution dans le temps ;
- visualiser une relation entre deux variables.

⇒ L'objectif est donc de construire progressivement une base d'analyse exploitable à partir de données brutes.

Importer et inspecter les données

Importer et inspecter les données

Avant de manipuler une base, il faut d'abord comprendre ce qu'elle contient.

Dans ce bloc, nous allons apprendre à :

- importer une base de données dans R ;
- identifier l'unité d'observation ;
- examiner les variables disponibles ;
- repérer les premières difficultés ;
- préparer le passage vers le nettoyage des données.

La base utilisée dans la séance

Nous allons travailler sur la base **CO2 and Greenhouse Gas Emissions** produite par Our World in Data.

Cette base contient des données annuelles par pays sur :

- les émissions de CO₂ ;
- les émissions de CO₂ par habitant ;
- la population ;
- le PIB ;
- les émissions cumulées ;
- les sources d'émissions liées à l'énergie.

→ L'unité d'observation principale est donc : pays $\times$ année

Question à toujours se poser : *Que représente une ligne de la base ?*

Charger les packages utiles

Avant de commencer, on charge le package `tidyverse`.

```
library(tidyverse)
```

Le package charge un ensemble de packages que nous avons précédemment vus.

Bonne pratique : Toujours placer le chargement des packages au début du script.

Avant d'importer : créer un projet R

Avant de charger les données, il est recommandé de créer un **projet RStudio**.

Un projet permet de regrouper au même endroit :

- les scripts R ;
- les données brutes et nettoyées ;
- les graphiques et tableaux ;
- les documents produits avec RMarkdown.

Un projet R permet d'utiliser des chemins relatifs et de rendre l'analyse plus facile à reproduire.

Organiser son dossier de travail

Une organisation simple suffit pour commencer.

```
projet_co2/  
├─ data/  
│   ├── raw/  
│   └─ clean/  
├─ scripts/  
├─ outputs/  
│   ├── figures/  
│   └─ tables/  
└─ projet_co2.Rproj
```

Les données brutes ne doivent pas être modifiées directement : on les importe, on les nettoie, puis on sauvegarde une version propre.

Importer une base CSV

On importe la base avec la fonction `read_csv()`.

```
co2_raw <- read_csv("DATA/owid-co2-data.csv")
```

Cette commande crée un objet appelé `co2_raw`.

Quelques remarques :

- `read_csv()` est adapté aux fichiers séparés par des virgules ;
- le chemin indique où se trouve le fichier ;
- l'objet importé est stocké dans l'environnement R ;
- le nom `raw` indique que la base n'a pas encore été modifiée.

Bonne pratique : Garder une version brute de la base permet de toujours pouvoir revenir aux données initiales.

Premières commandes d'inspection

Une fois la base importée, on commence par regarder sa structure.

```
glimpse(co2_raw)
```

On peut aussi utiliser :

```
dim(co2_raw)  
names(co2_raw)  
head(co2_raw)
```

Ces commandes permettent de répondre à trois questions simples :

- combien y a-t-il de lignes et de colonnes ?
- quelles sont les variables disponibles ?
- à quoi ressemblent les premières observations ?

Avant toute transformation, toujours inspecter la base.

Identifier l'unité d'observation

Une étape essentielle consiste à comprendre ce que représente une ligne.

Dans notre cas, une ligne correspond en principe à :
un pays ou une zone géographique $\times$ une année

On peut le vérifier avec quelques commandes simples :

```
co2_raw %>%  
  select(country, iso_code, year, co2, co2_per_capita) %>%  
  head()
```

La base contient-elle uniquement des pays, ou aussi des régions et des agrégats ?

Pays, régions et agrégats

Les bases internationales contiennent souvent à la fois :

- des pays ;
- des continents ;
- des groupes de pays ;
- des agrégats mondiaux.

Dans notre base, on peut identifier certains agrégats :

```
co2_raw %>%  
  filter(is.na(iso_code)) %>%  
  distinct(country) %>%  
  arrange(country)
```

Attention : Ne pas distinguer pays et agrégats peut conduire à des statistiques descriptives trompeuses.

Explorer les années disponibles

Avant de définir un échantillon, il faut connaître la période couverte par la base.

```
co2_raw %>%  
  summarise(  
    first_year = min(year, na.rm = TRUE),  
    last_year = max(year, na.rm = TRUE)  
  )
```

On peut aussi regarder le nombre d'observations par année :

```
co2_raw %>%  
  count(year) %>%  
  arrange(year)
```

La couverture temporelle est-elle homogène entre les pays ?

Explorer les variables clés

Pour notre analyse, certaines variables sont particulièrement importantes.

```
co2_raw %>%  
  select(country, iso_code, year, population, gdp, co2, co2_per_c  
  glimpse()
```

Explorer les variables clés

Variables utiles :

- `country` : nom du pays ou de la zone ;
- `iso_code` : code pays ;
- `year` : année ;
- `population` : population ;
- `gdp` : PIB ;
- `co2` : émissions annuelles de CO₂ ;
- `co2_per_capita` : émissions de CO₂ par habitant.

L'objectif est d'identifier les variables nécessaires pour construire la base d'analyse.

Vérifier les valeurs manquantes

Les valeurs manquantes sont très fréquentes dans les données empiriques.

On peut les compter pour quelques variables importantes :

```
co2_raw %>%  
  summarise(  
    missing_population = sum(is.na(population)),  
    missing_gdp = sum(is.na(gdp)),  
    missing_co2 = sum(is.na(co2)),  
    missing_co2_pc = sum(is.na(co2_per_capita))  
  )
```

Attention : Supprimer les valeurs manquantes n'est jamais une opération neutre : cela modifie l'échantillon étudié.

Vérifier les doublons

Dans une base pays-années, on s'attend à avoir au plus une observation par pays et par année.

On peut vérifier cette structure avec :

```
co2_raw %>%  
  count(country, year) %>%  
  filter(n > 1)
```

Si cette commande renvoie des lignes, cela signifie qu'il existe plusieurs observations pour un même pays et une même année.

Bonne pratique : Toujours vérifier que la structure de la base correspond à l'unité d'observation attendue.

Préparer la suite

À ce stade, nous avons commencé à comprendre la base.

Nous savons :

- quelles variables sont disponibles ;
- quelle est l'unité d'observation ;
- quelle période est couverte ;
- quelles variables contiennent des valeurs manquantes ;
- si la base contient des pays, des régions ou des agrégats.

La prochaine étape consiste à construire une base plus propre pour l'analyse.

Construire une base d'analyse

De la base brute à la base d'analyse

Une base brute contient souvent trop d'informations pour répondre directement à une question empirique.

Construire une base d'analyse consiste à :

- sélectionner les observations utiles ;
- garder les variables nécessaires ;
- créer de nouvelles variables ;
- traiter les valeurs manquantes ;
- vérifier que l'unité d'observation est correcte.

Une base d'analyse est une version structurée de la base brute, construite pour répondre à une question de recherche précise.

Définir une question empirique simple

Dans cette séance, nous partons d'une question descriptive :

Comment les émissions de CO2 par habitant évoluent-elles selon les pays depuis 1990 ?

Pour répondre à cette question, nous avons besoin :

- d'une unité d'observation pays-année ;
- d'une période d'analyse ;
- d'une variable d'émissions par habitant ;
- de quelques variables de comparaison ;
- d'un échantillon cohérent.

Avant de nettoyer une base, il faut savoir ce que l'on cherche à analyser.

Sélectionner les observations utiles

On commence par restreindre la base à l'échantillon souhaité.

```
co2_analysis <- co2_raw %>%  
  filter(  
    year >= 1990,  
    !is.na(iso_code)  
  )
```

Ici, on garde :

- les observations depuis 1990 ;
- les observations avec un code pays ;
- donc principalement les pays, et non les grands agrégats.

Attention : Le choix de l'échantillon doit toujours être explicite et justifié.

Sélectionner les variables utiles

Une fois l'échantillon défini, on garde les variables nécessaires.

```
co2_analysis <- co2_raw %>%  
  filter(  
    year >= 1990,  
    !is.na(iso_code)  
  ) %>%  
  select(  
    country, iso_code, year,  
    population, gdp,  
    co2, co2_per_capita  
  )
```

Cela permet d'obtenir une base plus lisible.

Bonne pratique : Ne garder que les variables utiles facilite la lecture du code et réduit les risques d'erreur.

Créer de nouvelles variables

Les variables directement disponibles ne suffisent pas toujours.

On peut créer de nouveaux indicateurs avec `mutate()`.

```
co2_analysis <- co2_analysis %>%  
  mutate(  
    gdp_per_capita = gdp / population,  
    log_gdp_pc = log(gdp_per_capita),  
    log_co2_pc = log(co2_per_capita)  
  )
```

Ici, on crée :

- le PIB par habitant ;
- le log du PIB par habitant ;
- le log des émissions de CO₂ par habitant.

Attention : Le logarithme n'est défini que pour des valeurs strictement positives.

Traiter les valeurs manquantes

Avant d'utiliser une variable, il faut vérifier si elle contient des valeurs manquantes.

```
co2_analysis %>%  
  summarise(  
    missing_population = sum(is.na(population)),  
    missing_gdp = sum(is.na(gdp)),  
    missing_co2_pc = sum(is.na(co2_per_capita))  
  )
```

Traiter les valeurs manquantes

On peut ensuite restreindre la base aux observations exploitables :

```
co2_analysis <- co2_analysis %>%  
  filter(  
    !is.na(population),  
    !is.na(gdp),  
    !is.na(co2_per_capita)  
  )
```

Attention : Supprimer des valeurs manquantes modifie l'échantillon → ce choix doit être documenté.

Vérifier la base obtenue

Après nettoyage, il faut toujours vérifier la base produite.

```
glimpse(co2_analysis)
```

```
co2_analysis %>%  
  summarise(  
    n_obs = n(),  
    n_countries = n_distinct(country),  
    first_year = min(year),  
    last_year = max(year)  
  )
```

Vérifier la base obtenue

Ces commandes permettent de vérifier :

- le nombre d'observations ;
- le nombre de pays ;
- la période couverte ;
- le type des variables.

Après chaque étape importante de nettoyage, vérifier que la base obtenue correspond à ce que l'on attend.

Une chaîne complète de traitement

On peut regrouper les étapes dans une seule chaîne lisible.

```
co2_analysis <- co2_raw %>%  
  filter(  
    year >= 1990,  
    !is.na(iso_code),  
    !is.na(population),  
    !is.na(gdp),  
    !is.na(co2_per_capita),  
    population > 0,  
    gdp > 0,  
    co2_per_capita > 0  
  ) %>%  
  mutate(  
    gdp_per_capita = gdp / population,  
    log_gdp_pc = log(gdp_per_capita),  
    log_co2_pc = log(co2_per_capita)  
  ) %>%  
  select(  
    country, iso_code, year,  
    population, gdp, gdp per capita,
```

Documenter ses choix

Construire une base d'analyse implique toujours des décisions.

Exemples :

- pourquoi commencer en 1990 ?
- pourquoi supprimer les observations sans code pays ?
- pourquoi exclure les valeurs manquantes ?
- pourquoi utiliser des variables par habitant ?
- pourquoi passer certaines variables en logarithme ?

Ces choix doivent être visibles dans le code et expliqués dans le travail de recherche.

Un script ne doit pas seulement produire un résultat : il doit aussi rendre compréhensible le chemin qui mène au résultat.

Vers l'analyse descriptive...

Nous disposons maintenant d'une base d'analyse plus propre.

Elle permet de produire :

- des statistiques descriptives ;
- des comparaisons entre pays ;
- des évolutions temporelles ;
- des graphiques exploratoires ;
- une première analyse économétrique simple.

Utiliser cette base pour produire des faits stylisés clairs et interprétables.

Statistiques descriptives

Pourquoi commencer par des statistiques descriptives ?

Avant d'estimer un modèle, il faut comprendre les données.

Les statistiques descriptives permettent de répondre à des questions simples :

- combien d'observations sont disponibles ?
- combien de pays sont couverts ?
- quelle est la période étudiée ?
- quelles sont les valeurs moyennes et médianes ?
- les variables contiennent-elles beaucoup de valeurs manquantes ?
- existe-t-il des valeurs extrêmes ?

Les statistiques descriptives sont la première étape de toute analyse empirique sérieuse.

Décrire l'échantillon d'analyse

On commence par décrire la base dans son ensemble.

```
co2_analysis %>%  
  summarise(  
    n_obs = n(),  
    n_countries = n_distinct(country),  
    first_year = min(year, na.rm = TRUE),  
    last_year = max(year, na.rm = TRUE)  
  )
```

Cette étape permet de vérifier :

- la taille de la base ;
- le nombre de pays observés ;
- la période couverte ;
- la cohérence avec l'échantillon souhaité.

Toujours décrire l'échantillon avant de présenter des résultats.

Décrire une variable quantitative

On peut résumer une variable avec quelques indicateurs simples.

```
co2_analysis %>%  
  summarise(  
    mean_co2_pc = mean(co2_per_capita, na.rm = TRUE),  
    median_co2_pc = median(co2_per_capita, na.rm = TRUE),  
    sd_co2_pc = sd(co2_per_capita, na.rm = TRUE),  
    min_co2_pc = min(co2_per_capita, na.rm = TRUE),  
    max_co2_pc = max(co2_per_capita, na.rm = TRUE)  
  )
```

Ces statistiques donnent une première idée :

- du niveau moyen de la variable ;
- de la dispersion ;
- de l'existence éventuelle de valeurs extrêmes.

Attention : La moyenne peut être fortement influencée par quelques observations extrêmes.

Comparer moyenne et médiane

La comparaison entre moyenne et médiane est souvent informative.

```
co2_analysis %>%  
  summarise(  
    mean_co2_pc = mean(co2_per_capita, na.rm = TRUE),  
    median_co2_pc = median(co2_per_capita, na.rm = TRUE)  
  )
```

Si la moyenne est très supérieure à la médiane, cela peut indiquer que :

- la distribution est asymétrique ;
- quelques pays ont des émissions très élevées ;
- la variable peut nécessiter une transformation logarithmique.

Les statistiques descriptives ne sont pas seulement techniques : elles aident à comprendre la structure des données.

Décrire plusieurs variables

On peut produire des statistiques descriptives pour plusieurs variables à la fois.

```
co2_analysis %>%  
  summarise(  
    across(  
      c(co2_per_capita, gdp_per_capita, population),  
      list(  
        mean = ~ mean(.x, na.rm = TRUE),  
        median = ~ median(.x, na.rm = TRUE),  
        sd = ~ sd(.x, na.rm = TRUE)  
      )  
    )  
  )  
)
```

La fonction `across()` permet d'appliquer les mêmes calculs à plusieurs variables.

Bonne pratique : Éviter de répéter le même code plusieurs fois : cela réduit les erreurs et facilite les modifications.

Statistiques par groupe

Dans une analyse empirique, on souhaite souvent comparer des groupes.

Par exemple, on peut calculer les émissions moyennes par pays :

```
country_stats <- co2_analysis %>%  
  group_by(country) %>%  
  summarise(  
    mean_co2_pc = mean(co2_per_capita, na.rm = TRUE),  
    mean_gdp_pc = mean(gdp_per_capita, na.rm = TRUE),  
    n_years = n(),  
    .groups = "drop"  
  )
```

`group_by()` définit les groupes ; `summarise()` calcule les indicateurs pour chaque groupe.

Classer les observations

Après avoir calculé des statistiques par pays, on peut classer les résultats.

```
country_stats %>%  
  arrange(desc(mean_co2_pc)) %>%  
  select(country, mean_co2_pc, mean_gdp_pc) %>%  
  head(10)
```

Ce type de commande permet d'identifier rapidement :

- les pays avec les émissions les plus élevées ;
- les pays atypiques ;
- les observations à vérifier plus attentivement.

Attention : Un classement descriptif ne constitue pas une explication causale.

Évolution dans le temps

On peut également produire des statistiques descriptives par année.

```
year_stats <- co2_analysis %>%  
  group_by(year) %>%  
  summarise(  
    mean_co2_pc = mean(co2_per_capita, na.rm = TRUE),  
    median_co2_pc = median(co2_per_capita, na.rm = TRUE),  
    n_countries = n_distinct(country),  
    .groups = "drop"  
  )
```

Cette table permet de suivre :

- l'évolution moyenne des émissions ;
- la couverture de la base au cours du temps ;
- les ruptures ou changements de tendance.

Cette table pourra ensuite être utilisée pour produire un graphique d'évolution temporelle.

Repérer les valeurs extrêmes

Certaines observations peuvent fortement influencer les résultats.

```
co2_analysis %>%  
  arrange(desc(co2_per_capita)) %>%  
  select(country, year, co2_per_capita, gdp_per_capita) %>%  
  head(10)
```

Cette étape permet de repérer :

- des pays très atypiques ;
- des erreurs possibles de mesure ;
- des observations qui méritent une vérification ;
- des cas intéressants à discuter.

Bonne pratique : Une valeur extrême ne doit pas être supprimée automatiquement : elle doit d'abord être comprise.

Produire une table claire

Pour un article, un mémoire ou une thèse, les statistiques descriptives doivent être lisibles.

```
desc_table <- co2_analysis %>%  
  summarise(  
    Observations = n(),  
    Pays = n_distinct(country),  
    `CO2 par habitant moyen` = mean(co2_per_capita, na.rm = TRUE),  
    `PIB par habitant moyen` = mean(gdp_per_capita, na.rm = TRUE)  
  )  
  
desc_table
```


D'autres packages utiles

Le tidyverse permet déjà de produire des statistiques descriptives simples.

Mais certains packages facilitent la production de tableaux plus propres et plus rapides :

- `table1` : tableaux descriptifs par groupes ;
- `skimr` : résumé rapide d'une base de données ;
- `summarytools` : statistiques descriptives détaillées ;
- `gtsummary` : tableaux descriptifs prêts pour rapports/articles ;
- `modelsummary` : tableaux de statistiques et résultats de régression.

Les erreurs classiques

Pourquoi parler des erreurs ?

Les erreurs dans le traitement des données sont fréquentes.

Elles peuvent venir :

- d'une mauvaise compréhension de la base ;
- d'un mauvais choix d'échantillon ;
- d'un problème d'unité ;
- de valeurs manquantes mal traitées ;
- d'une confusion entre corrélation et causalité ;
- d'un code difficile à relire.

Une partie importante du travail empirique consiste à éviter les erreurs silencieuses.

Les erreurs à éviter

Dans un travail empirique, certaines erreurs reviennent très souvent.

- Ne pas vérifier l'unité d'observation de la base.
- Mélanger pays, régions et agrégats sans s'en rendre compte.
- Supprimer les valeurs manquantes sans regarder quelles observations disparaissent.
- Oublier les unités des variables : total, par habitant, pourcentage, logarithme.
- Créer une variable en logarithme sans vérifier les valeurs nulles ou négatives.

Les erreurs à éviter

- Agréger les données sans réfléchir au niveau pertinent d'analyse.
- Écraser la base brute au lieu de créer une base nettoyée.
- Ne pas vérifier la base après un filtre, une jointure ou une transformation.
- Confondre corrélation descriptive et relation causale.
- Produire un code difficile à relire ou à reproduire.

La qualité d'une analyse empirique dépend autant des choix de nettoyage que des modèles estimés.

Des graphiques utiles pour économistes avec GGPLOT2

Des graphiques utiles pour économistes

Les graphiques sont une étape importante du travail empirique.

Ils permettent de :

- comprendre la distribution des variables ;
- repérer des valeurs extrêmes ;
- suivre des évolutions dans le temps ;
- comparer des groupes ;
- visualiser des relations entre variables ;
- formuler des hypothèses pour l'analyse économétrique.

Un bon graphique ne remplace pas l'analyse économétrique, mais il aide à comprendre les données.

La logique de ggplot

Un graphique avec ggplot2 se construit par couches successives (*the grammar of graphics*).

```
ggplot(data = co2_analysis, aes(x = co2_per_capita)) +  
  geom_histogram() +  
  labs(  
    title = "Titre du graphique",  
    x = "Nom de l'axe horizontal",  
    y = "Nom de l'axe vertical"  
  ) +  
  theme_minimal()
```


La logique de ggplot

Les éléments principaux sont :

- `data` : la base utilisée ;
- `aes()` : les variables représentées ;
- `geom_*()` : le type de graphique ;
- `labs()` : les titres et labels ;
- `theme_*()` : l'apparence du graphique.

Étudier une distribution

Un histogramme permet de visualiser la distribution d'une variable quantitative.

```
ggplot(co2_analysis, aes(x = co2_per_capita)) +  
  geom_histogram(bins = 40) +  
  labs(  
    title = "Distribution des émissions de CO2 par habitant",  
    x = "CO2 par habitant",  
    y = "Nombre d'observations"  
  ) +  
  theme_minimal()
```

Ce graphique permet de repérer :

- la concentration des observations ;
- l'asymétrie de la distribution ;
- la présence de valeurs extrêmes.

La variable est-elle distribuée de manière symétrique ?

Distribution : pourquoi utiliser le log ?

Certaines variables économiques ont des distributions très asymétriques.

C'est souvent le cas pour :

- le PIB ;
- la population ;
- les revenus ;
- les émissions de CO₂.

Distribution : pourquoi utiliser le log ?

On peut alors représenter la variable en logarithme.

```
ggplot(co2_analysis, aes(x = log_co2_pc)) +  
  geom_histogram(bins = 40) +  
  labs(  
    title = "Distribution du log des émissions de CO2 par habitant",  
    x = "log(CO2 par habitant)",  
    y = "Nombre d'observations"  
  ) +  
  theme_minimal()
```

Attention : Le logarithme n'est défini que pour les valeurs strictement positives.

Étudier une évolution temporelle

On commence par construire une base agrégée par année.

```
co2_year <- co2_analysis %>%  
  group_by(year) %>%  
  summarise(  
    mean_co2_pc = mean(co2_per_capita, na.rm = TRUE),  
    median_co2_pc = median(co2_per_capita, na.rm = TRUE),  
    .groups = "drop"  
  )
```

Cette base permet de suivre l'évolution moyenne des émissions de CO2 par habitant.

Avant de faire un graphique, il faut souvent construire la table correspondant au niveau d'analyse souhaité.

Série temporelle

Une courbe permet de représenter l'évolution d'une variable dans le temps.

```
ggplot(co2_year, aes(x = year, y = mean_co2_pc)) +  
  geom_line() +  
  labs(  
    title = "Évolution moyenne des émissions de CO2 par habitant",  
    x = "Année",  
    y = "CO2 par habitant moyen"  
  ) +  
  theme_minimal()
```

Ce graphique permet d'observer :

- une tendance générale ;
- des ruptures éventuelles ;
- des périodes de hausse ou de baisse ;
- des changements dans la dynamique temporelle.

Moyenne ou médiane ?

La moyenne et la médiane ne donnent pas toujours la même information.

```
ggplot(co2_year, aes(x = year)) +  
  geom_line(aes(y = mean_co2_pc), linetype = "solid") +  
  geom_line(aes(y = median_co2_pc), linetype = "dashed") +  
  labs(  
    title = "Émissions de CO2 par habitant : moyenne et médiane",  
    x = "Année",  
    y = "CO2 par habitant"  
  ) +  
  theme_minimal()
```

Visualiser une relation entre deux variables

On construit d'abord une base au niveau pays.

```
country_summary <- co2_analysis %>%  
  group_by(country) %>%  
  summarise(  
    mean_gdp_pc = mean(gdp_per_capita, na.rm = TRUE),  
    mean_co2_pc = mean(co2_per_capita, na.rm = TRUE),  
    .groups = "drop"  
  ) %>%  
  filter(mean_gdp_pc > 0, mean_co2_pc > 0)
```

Cette base permet de comparer les pays selon leur niveau moyen de développement et leurs émissions moyennes.

PIB par habitant et émissions de CO2

Un nuage de points permet de visualiser la relation entre deux variables quantitatives.

```
ggplot(country_summary, aes(x = mean_gdp_pc, y = mean_co2_pc)) +  
  geom_point(alpha = 0.6) +  
  scale_x_log10() +  
  scale_y_log10() +  
  geom_smooth(method = "lm", se = FALSE) +  
  labs(  
    title = "PIB par habitant et émissions de CO2 par habitant",  
    subtitle = "Moyennes par pays depuis 1990",  
    x = "PIB par habitant, échelle log",  
    y = "CO2 par habitant, échelle log"  
  ) +  
  theme_minimal()
```

Chaque point représente un pays. La droite résume la relation moyenne entre les deux variables.

Corrélation, visualisation et causalité

Le graphique précédent peut suggérer une relation entre développement économique et émissions de CO₂

Mais il ne permet pas de conclure que :
PIB par habitant → émissions de CO₂ par habitant

Plusieurs problèmes peuvent exister :

- variables omises ;
- causalité inverse ;
- différences structurelles entre pays ;
- trajectoires historiques différentes ;
- effets de composition sectorielle.

Un graphique décrit une relation. Il ne suffit pas à identifier un effet causal.

Merci beaucoup pour votre
attention !

Maintenant discutons de vos
recherches !